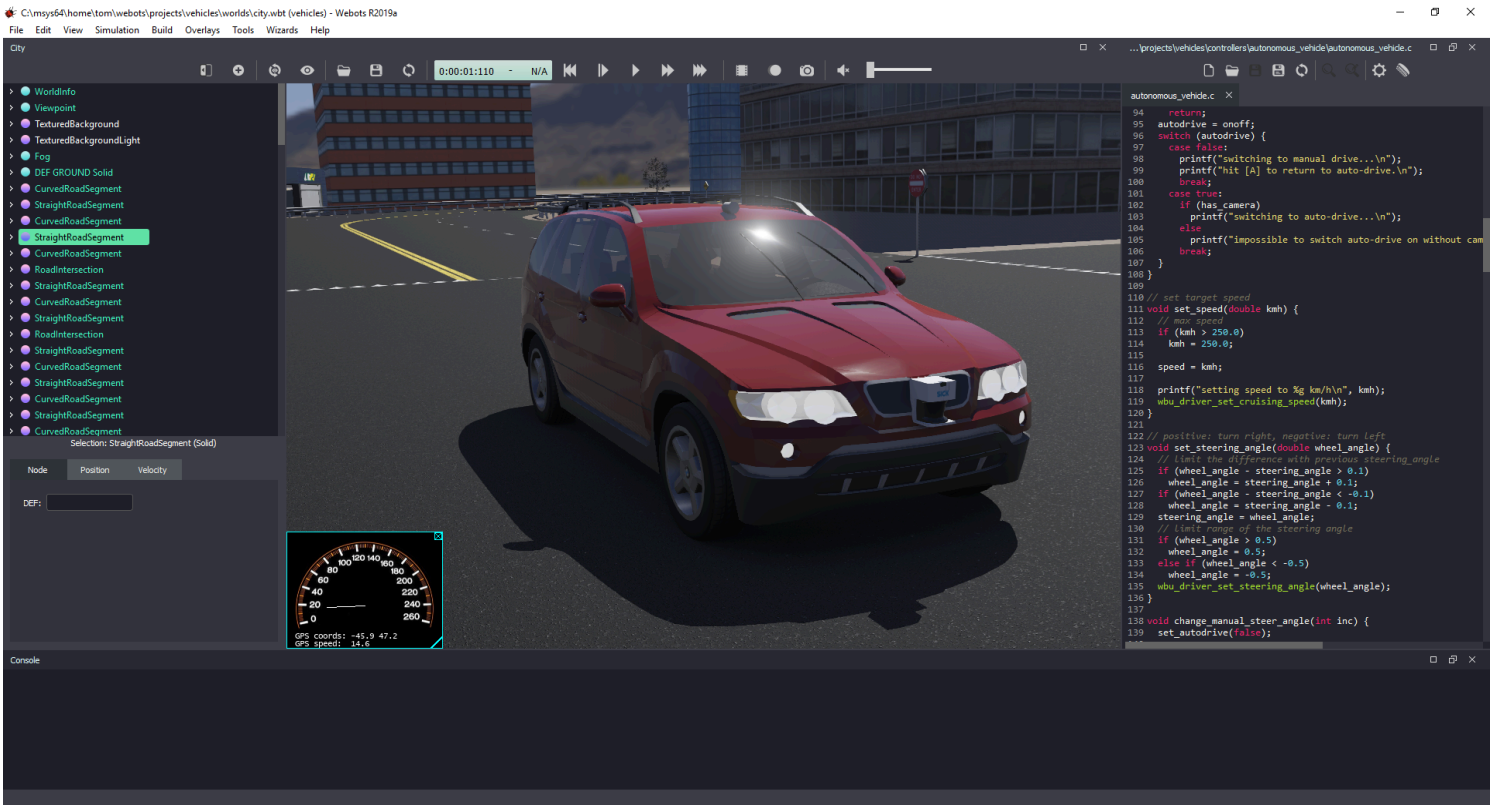
WEBOTS TUTORIAL HANDBOOK



Introduction

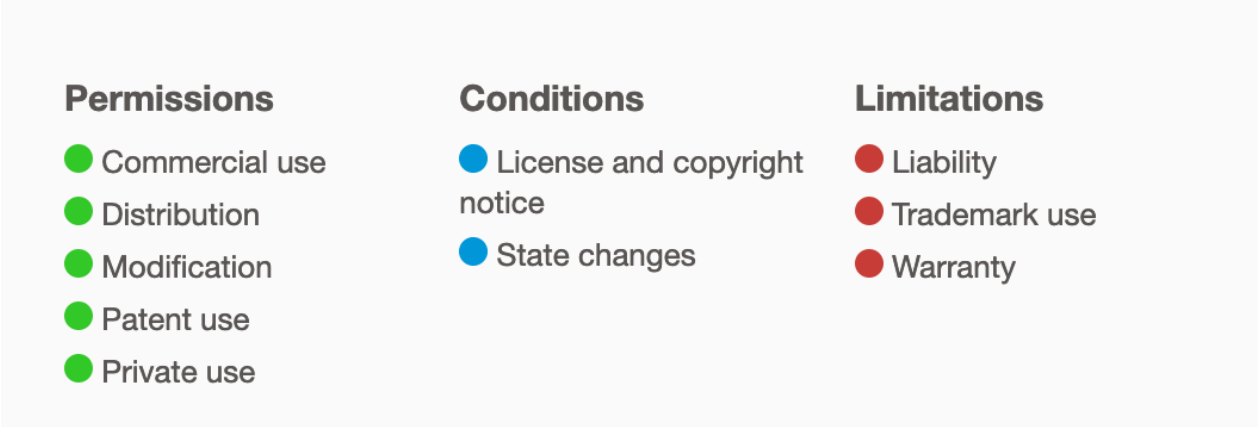
Webots is a development environment used to model, program, and simulate mobile robots. It uses the **ODE (Open Dynamics Engine)** for physics and allows users to control robots using C, C++, Python, Java, or MATLAB.

Key Features

- **Cross-Platform:** Windows, Linux, macOS.
- **High Fidelity:** Realistic physics, friction, and mass distribution.
- **Sensor Simulation:** Lidar, Cameras, GPS, IMU, etc.
- **ROS Integration:** Native support for ROS and ROS2.

I. Licence

Apache 2.0 license agreement



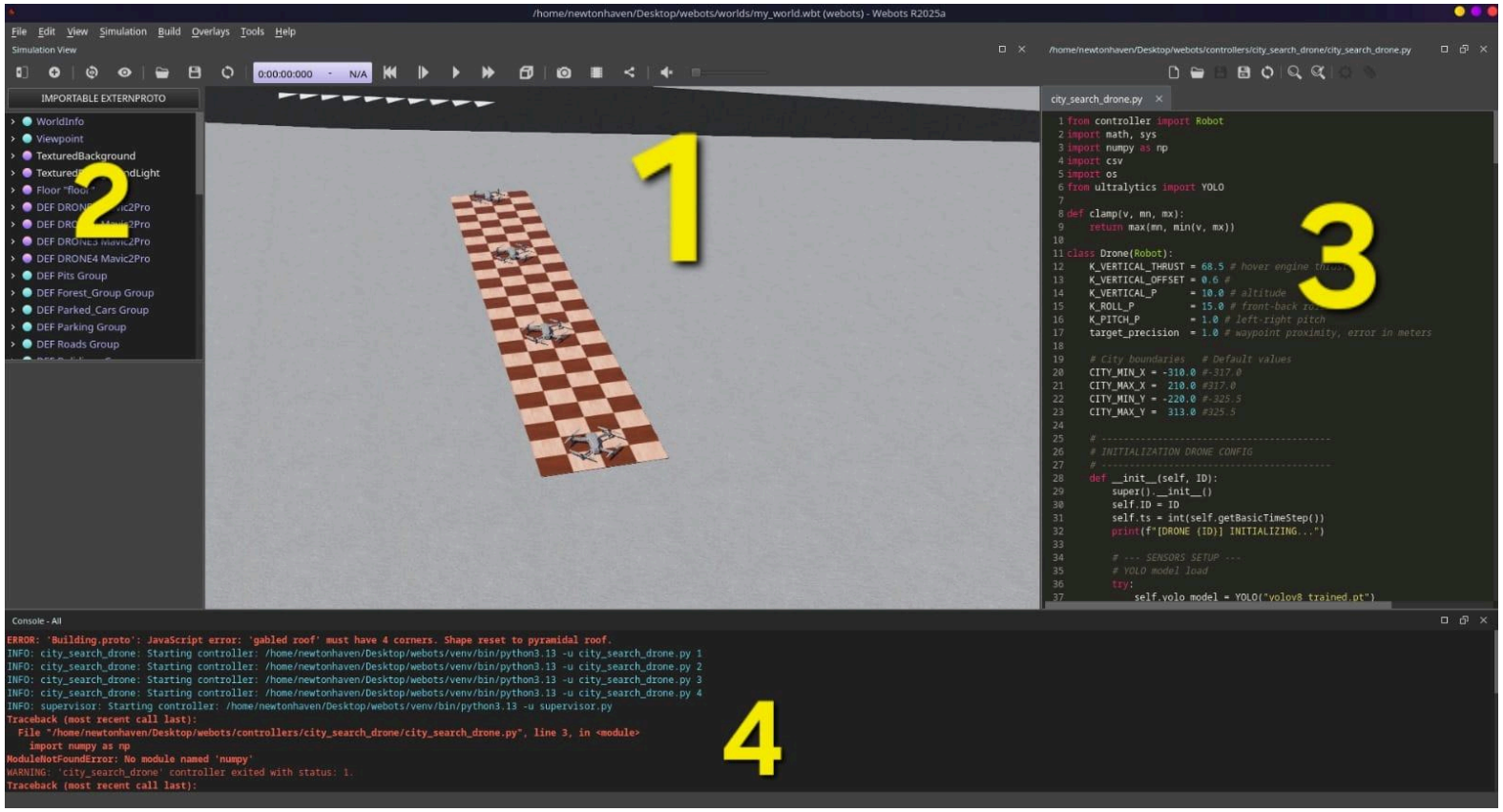
II. Installation

System Requirements

- **OS:** Windows 10/11 (64-bit), Ubuntu 20.04/22.04, or macOS 11+.
- **Graphics:** NVIDIA or AMD graphics card is recommended, and OpenGL 3.3 support is required.

OS	Installation Method
Windows & MacOS	Download from the official GitHub repository releases (stable version advised) and run the installer. https://github.com/cyberbotics/webots/releases
Ubuntu / Debian	Run command: `sudo snap install webots`
Arch Linux	Run command: `paru -S webots` OR `yay -S webots`

III. The Interface & Controls



UI Element	Description
1- 3D View	The main window where the simulation renders. (!) In Linux, If screen scale setting is changed from default in system settings (of operating system), may cause an offset problem with 3D Screen of Webots.
2- Scene Tree	Webots worlds are built using **Nodes**. The most important concept is the parent-child relationship. • **WorldInfo:** Defines gravity, physics step size, and basic global parameters. • **Viewpoint:** Where the camera is positioned when the world loads. • **Background:** The skybox or horizon color. • **Solid:** A physical object with mass and collision bounds. • **Robot:** A special type of Solid that includes a controller field. Scene Tree can be edited via any text editor for quicker editing, grouping.
3 -Text Editor	Built-in IDE for writing controller code (Other IDEs also can be used).
4- Console	Displays compilation errors and print() outputs.

5- Top Bar Controls



1- Save World.

2- Reload World. [Necessary after script change]

3- Timer: Time past **in** simulation.

[EVERY CHANGE IN WORLD MUST BE DONE IN 0:00:00 TIME THEN SAVED. OTHERWISE IT WILL BE RESETED.]

4- Speed of Simulation: How fast the simulation is running. (Below 1x is slower than real life)

5- Rewind to the Start: Doesn't reload the world, just starts the current simulation from the beginning. Script changes don't affect.

6- Run / Pause.

7- Simulation Speed: Controls how fast the simulation runs.Depends on computing power.

8- Build-in Screen Recorder and Screenshot: Video/Photos of 3D View Screen of Webots.

IV. File Structure of Webots Projects

To keep simulations organized and ensure Webots can automatically find your scripts, physics plugins, and 3D models, every project must follow a strict directory hierarchy.

Plaintext

```
my_project/
│
├── worlds/               # Contains your simulation environments (.wbt files and forest folder)
│   ├── my_map.wbt
│
├── controllers/          # Contains all robot and supervisor scripts
│   ├── my_drone_controller/
│   │   └── my_drone_controller.py    # example script
│
├── protos/               # Custom robot or object definitions (.proto files)
│   └── CustomDrone.proto
│
└── plugins/              # Custom C/C++ plugins
    └── physics/
        └── wind_physics/
            └── wind_physics.c
```

---

v. **Building and Editing the World** (OpenStreetMap Importer)

Webots worlds can be generated from OpenStreetMap and any part of the world using the importer script. Official documentation.

1st STEP- Open OpenStreetMap Exporter: `https://www.openstreetmap.org/export#map=11/40.7837/-74.0245`

[Do not use more than a few square kilometers if you want to be able to run your simulation in real-time]

2nd STEP- Then install the Python `lxml`, `pyproj`, `shapely` and `webcolors` modules.

OS	**Command**

Linux	`sudo apt-get install python3-pip`
	`python3 -m pip install --upgrade pip`
	`python3 -m pip install lxml pyproj shapely webcolors configparser --user`

MacOS	`python3 -m pip install --upgrade pip` `python3 -m pip install lxml pyproj shapely webcolors configparser --user`
Windows	`pip install  lxml pyproj shapely webcolors`

3rd STEP- You should use the `importer.py` Python script to generate the `myMap.wbt` for webots simulation world from the `myMap.osm` file:

Script location: `"Webots Location"/resources/osm_importer`

(!) Script should be in the same location with the input file, or path should be included into bash script.

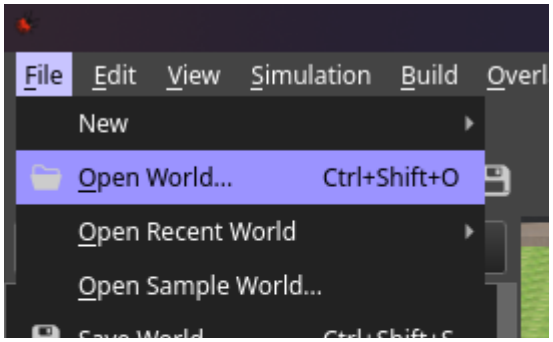
Usage:
`python3 - importer.py --input="input file name" --output="desired output name"`

Example:
`python3 importer.py --input=map.osm --output=myMap.wbt`

Some extra folders such as `forest` can be generated in the target directory depending on the importer arguments.

Arguments: https://cyberbotics.com/doc/automobile/openstreetmap-importer#arguments

myMap.wbt now can be opened with WeBots.



IF OSM USED MUST DO:
Imported maps will have their own real-life gps data. To fix this:

```
1- Open WordInfo Node:
2- gpsCoordinateSystem -> Local
```

Also seek trouble shooting part for possible simulation problems.

World / Environment Editing

EVERY CHANGE IN WORLD MUST BE DONE IN 0:00:00 TIME THEN SAVED. OTHERWISE IT WILL BE RESETED.

Every object in the Webots world is has their own node. Because of that every object can be edited, unnecessary parameters can be deleted.

Every node can be edited via any text editor. This is advised for quicker editing, grouping etc.

VI. Sensors, Physics, and PROTOs

Webots supports an extensive library of over 100 robots (including real-world models like the Mavic PRO 2 and custom imaginary builds) and sensors. You can browse the full catalog at:

https://webots.cloud/proto

Common Sensors and Setup

- **Camera:** Returns an array of pixels.
  - *Usage:* Object detection, line following.
  - *Note:* High resolution slows down simulation drastically.
- **Lidar:** Returns a point cloud or array of distances.
  - *Usage:* SLAM (Simultaneous Localization and Mapping).
- **GPS/InertialUnit:** Used for odometry.

Physics Plugins

Standard controllers only control robots. If you need to simulate "environmental physics" (e.g., wind affecting a drone, or a conveyor belt moving items), you need a **Physics Plugin**.

- This requires C/C++.
- It hooks directly into the ODE (Open Dynamics Engine) physics step.

https://github.com/cyberbotics/webots/tree/released/resources/projects/plugins/physics

- [ ] You can add a new plugin, or edit the existing plugin, by using the menu Tools / Edit Physics Plugin.
- [ ] After a physics plugin was created it must be associated with the current ".wbt" file.

- [ ] This can be done in the Scene Tree: the WorldInfo node has a field called physics which indicates the name of the physics plugin associated with the current world.
- [ ] Select the WorldInfo.physics field, then hit the Select... button.

PROTOs (Prototypes)

If you create a custom robot, you shouldn't rebuild it node-by-node every time. You save it as a .proto file. This acts like a "Class" in programming, while the robot in the scene is an "Instance."

VII. ROS2 and Supervisor Nodes

Supervisor Controller

A **Supervisor** is a special robot controller with "God Mode" privileges. It can:

- Reset the simulation.
- Teleport objects (perfect for Reinforcement Learning resets).
- Record videos.
- Track the exact position of objects (ground truth).

How to enable:

1. In the Robot node, set the supervisor field to TRUE.
2. Import the Supervisor class: from controller import Supervisor.

ROS2 Integration

- **webots_ros2:** This is the official package. https://github.com/cyberbotics/webots_ros2
- Instead of writing a Webots-specific controller, you write a standard ROS2 node.
- The webots_ros2_driver acts as the bridge, converting Webots sensor data into ROS topics (e.g., /scan, /cmd_vel, /camera/image_raw).

VIII. Troubleshooting & Best Practices

Common Error	Solution
Robot moves through walls	Check the boundingObject in the Scene Tree; if it is NULL, the object has no physical presence.
Robot is shaking violently	The basic TimeStep might be too large, or the robot's mass is too small relative to the forces.

Simulation is lagging	Disable shadows, reduce Camera/Lidar resolution, or increase the basicTimeStep.

Workflow Tips

1. **Use extern controllers:** You can run your Python script from your terminal (or PyCharm/VS Code) instead of inside Webots. This allows you to use Debuggers and standard Python environments (Anaconda/Virtualenv).
2. **Keep simple geometry:** Use simple shapes (Box, Cylinder) for collision (boundingObject) and complex meshes only for visuals (Shape). This saves computation power.

IX. Core Concepts & Code Examples

Example 1: Drone Motor Mixing

Python

```
# Calculate basic stabilizing inputs
alt_err = clamp(target_alt - z + K_VERTICAL_OFFSET, -1.0, 1.0)
vertical = K_VERTICAL_P * (alt_err ** 3)

# Add gyro (ra, pa) and desired movement (pitch_d, yaw_d)
roll_input  = K_ROLL_P  * clamp(roll,  -1, 1) + ra
pitch_input = K_PITCH_P * clamp(pitch, -1, 1) + pa + pitch_d
yaw_input   = yaw_d

# Mix inputs for each specific motor
fl = K_VERTICAL_THRUST + vertical - roll_input + pitch_input - yaw_input
fr = K_VERTICAL_THRUST + vertical + roll_input + pitch_input + yaw_input
rl = K_VERTICAL_THRUST + vertical - roll_input - pitch_input + yaw_input
rr = K_VERTICAL_THRUST + vertical + roll_input - pitch_input - yaw_input

# Apply to Webots motors
motor_fl.setVelocity(fl)
motor_fr.setVelocity(-fr) # Note: Opposite spinning direction
motor_rl.setVelocity(-rl)
motor_rr.setVelocity(rr)
```

Example 2: Accessing Camera Data & YOLO Integration

Webots cameras return a raw byte array. To use this with modern computer vision libraries like OpenCV or YOLO, it must be reshaped into a NumPy array.

Here is how you extract the image, pass it to YOLO, and verify the specific color of the target (a blue car).

Python

```
import numpy as np
from ultralytics import YOLO

# 1. Grab image from Webots Camera
raw_img = camera.getImage()
w, h = camera.getWidth(), camera.getHeight()

# 2. Reshape to standard image format (Height, Width, Channels)
img_arr = np.frombuffer(raw_img, dtype=np.uint8).reshape((h, w, 4))
img_bgr = img_arr[:, :, :3] # Drop Alpha channel, keep Blue, Green, Red

# 3. Pass to YOLO model
results = yolo_model(img_bgr, classes=[0], conf=0.65)

for r in results:
    for box in r.boxes:
        # Get coordinates of the bounding box center
        x1, y1, x2, y2 = map(int, box.xyxy[0])
        cx, cy = (x1 + x2) // 2, (y1 + y2) // 2

        # Check pixel color at the center (Webots uses BGR)
        b_val, g_val, r_val = img_bgr[cy, cx]

        # Target: Blue car (#0055ff -> R:0, G:85, B:255)
        diff = abs(r_val - 0) + abs(g_val - 85) + abs(b_val - 255)

        if diff < 255: # Tolerance threshold
            print("Target Confirmed!")
```

Example 3: Drone-to-Supervisor Radio Communication

To coordinate a swarm, drones need to talk to a central hub. Webots uses `Emitter` and `Receiver` nodes for this.

Sending a message (from the Drone):

```
# Format a string with gps data
msg = f"FOUND {current_x:.2f} {current_y:.2f}"
# Send as encoded bytes
emitter.send(msg.encode())
```

Receiving and parsing a message (in the Supervisor):

```
# Loop through all messages in the receiver queue
while receiver.getQueueLength() > 0:
    # Decode the message back to a string
    msg = receiver.getString()
    receiver.nextPacket() # Move to the next message

    # Parse the command
    if msg.startswith("FOUND"):
        parts = msg.split()
        target_x = float(parts[1])
        target_y = float(parts[2])
        print(f"Target located at {target_x}, {target_y}")
```

Example 4: Dynamically Drawing in the 3D World (Supervisor)

The Supervisor has a unique ability: modifying the Scene Tree while the simulation is running. This is incredibly useful for drawing debug lines, like drone flight trails.

This snippet creates an `IndexedLineSet` geometry out of thin air and injects it into the simulation's root node.

```
# Get the root of the Scene Tree
root_children = supervisor.getRoot().getField("children")

# Coordinates for the line segment
x1, y1, z1 = 0.0, 10.0, 0.0
x2, y2, z2 = 5.0, 10.0, 5.0
color = "1 0 0" # Red RGB

# Define the node using Webots VRML97-like syntax
shape_str = (
    f"Shape {{ "
    f"  appearance Appearance {{ material Material {{ emissiveColor
{color} }} }} "
```

```
    f"  geometry IndexedLineSet {{ "
    f"    coord Coordinate {{ point [ {x1} {y1} {z1}, {x2} {y2} {z2} ]
}} "
    f"    coordIndex [ 0, 1, -1 ] " # Draw from point 0 to point 1
    f"  }} "
    f"}}"
)

# Inject the new node into the world (-1 means add to the end of the
list)
root_children.importMFNodeFromString(-1, shape_str)
```

Extra: Additional WeBots Tutorials:

Official project based tutorials (Total Duration 4h)	https://cyberbotics.com/doc/guide/tutorials